



Practical: 1

Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort

Aim: To Implement and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort.

Algorithm:

i) Bubble sort Algorithm

Input : unsorted Array

Output : Sorted Array

Step 1 : for $i=0$ in condition $i < n$ then do

Step 2 : for $j=0$ in condition of $j < n-i-1$ then do

Step 3 : if $arr[j] > arr[j+1]$ then [swap]

swapping

$temp = arr[j]$

$arr[j] = arr[j+1]$

$arr[j+1] = temp$

Step 4 : Increment the inner loop of $i++$ iterat on

Step 5 : Increment the outer loop of $i++$ iterat on

Step 6 : Stop

(ii) Selection sort:

Input : unsorted Array

Output : Sorted Array

Step 1 : for $i=0$ in the condition $i < n$ then do

Step 2 : $min=i$ and inner for loop initialize $j=i+1$

Step 3 : in the condition $j < n$ then do if $arr[j] > arr[min]$ then $min=j$

Step 4 : increment the inner loop $j++$

Step 5 : if $(min \neq i)$ then swap($arr[i], arr[min]$)

Step 6 : increment the outer loop $i++$

Step 7 : Stop



(ii) Insertion sort :

Input : unsorted array
output : sorted array
step 1 : For $i=1$ in the condition $i \leq n$ then do
step 2 : $temp = arr[i]$ and $j = i-1$
step 3 : while $j \geq 0$ and $arr[j] > temp$ then
do
 $arr[j+1] = arr[j]$
 $j = j-1$
step 4 : initialize $arr[j+1] = temp$
step 5 : increment the outer loop $i++$
step 6 : STOP

(iv) Merge sort :

Input sort : unsorted Array
output : sorted Array
step 1 : check if array has more than one element
step 2 : Divide the array into two equal halves.
step 3 : Recursively sort the left half using
merge sort
step 4 : same for Right half using merge sort
step 5 : merge the two sorted halves into a
single sorted array
step 6 : stop.



(v) Quick sort

Input : unsorted array

Output : sorted array

Step 1 : select any element as a pivot

Step 2 : partition the array by placing
if $arr[i] \leq pivot$

$i++$

Step 3 : place pivot in its correct sorted
position

Step 4 : Recursively apply Quick sort

Step 5 : left sub array and Right sub-array

Step 7 : stop

Program:

1. Bubble sort, selection sort, Insertion sort

```
#include<iostream>
```

```
using namespace std;
```

```
class classname{
```

```
public:
```

```
void Func(){
```

```
int array1[100];
```

```
int input,i;
```

```
cout<<"Enter the size of array: ";
```

```
cin>>input;
```

```
array1[input];
```

```
cout<<"Enter the Elements in array:\n";
```

```
for(int i=0; i< input; i++){
```

```
    cin>>array1[i];
```

```
}
```

```
int n = sizeof(array1)/sizeof(array1[0]);
```

```
display(array1,input);
```

```
cout<<"\n\n";
```

```
Bubble(array1,input);
```



```
        cout<<"\n\n";
        selection(array1,input);
        cout<<"\n\n";
        insertion(array1,input);
        cout<<"\n\n";
//        quickSort(array1, 0, n-1);
    }
    void display(int array1[],int n){
        cout<<"Your Array is:\n";
        for(int i = 0; i<n; i++){
            cout<<array1[i]<<" ";
        }

    }
    void Bubble(int array1[],int n){
        cout<<"Bubble sorted array:\n";
        for(int i=0;i<n;i++){
            for(int j=0; j<n-i-1; j++){
                if(array1[j] > array1[j+1]){
                    int temp = array1[j];
                    array1[j] = array1[j+1];
                    array1[j+1] = temp;
                }
            }
        }
        display(array1,n);
    }
    void selection(int array1[],int n){
        cout<<"selection sort Array:\n";
        for(int i = 0; i<n; i++){
            int min = i;
            for(int j=i+1; j<n; j++){
                if(array1[j] < array1[min]){
                    min = j;
                }
            }
            if(min != i){
                int temp = array1[i];
                array1[i] = array1[min];
                array1[min] = temp;
            }
        }
        display(array1,n);
    }

    void insertion(int array1[],int n){
        cout<<"insertion sort Array:\n";
```



```
        for(int i=1;i<n;i++){
            int temp = array1[i];
            int j = i-1;
            while(j>=0 && array1[j]>temp){
                array1[j+1] = array1[j];
                j=j-1;
            }
            array1[j+1] = temp;
        }
        display(array1,n);
    }

int main(){
    classname obj;
    obj.Func();
    return 0;
}
```

2. Quick sort and merge sort:

```
#include<iostream>
```

```
using namespace std;
```

```
//----- Swap Function -----
```

```
void swap(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

```
//----- Quick Sort -----
```

```
int partition(int arr[], int start, int end) {
    int pivot = arr[end];
    int i = start - 1;
```

```
    for (int j = start; j < end; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
```

```
    swap(&arr[i + 1], &arr[end]);
    return i + 1;
```

```
}
```

```
void quickSort(int arr[], int start, int end) {
    if (start < end) {
```



```
int pi = partition(arr, start, end);

quickSort(arr, start, pi - 1);
quickSort(arr, pi + 1, end);
}
}

//----- Merge Sort -----
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int L[n1], R[n2];

    // Copy data to temporary arrays
    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];

    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;

    // Merge the temporary arrays
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        }
        else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }
    // Copy remaining elements
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }
}
```



```
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = (left + right) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

void display(int array1[], int n) {
    for (int i = 0; i < n; i++) {
        cout << array1[i] << " ";
    }
    cout << endl;
}

int main() {
    int arr[] = {9, 9, 7, 78, 5};
    int n = sizeof(arr) / sizeof(arr[0]);
    cout << "Original Array:\n";
    display(arr, n);
    //Quick Sort
    quickSort(arr, 0, n - 1);
    cout<<endl;
    cout<<"Quicksort sorted array: ";
    display(arr, n);
    // Merge Sort
    mergeSort(arr, 0, n - 1);
    cout<<endl;
    cout<<"mergesort sorted array: ";
    display(arr, n);
    return 0;
}
```

Output:

```
Enter the size of array: 5
Enter the Elements in array:
1 9 87 4 5
Your Array is:
1 9 87 4 5

Bubble sorted array:
Your Array is:
1 4 5 9 87

selection sort Array:
Your Array is:
1 4 5 9 87

insertion sort Array:
Your Array is:
1 4 5 9 87
```

Original Array:
9 9 7 78 5

Quicksort sorted array: 5 7 9 9 78

Mergesort sorted array: 5 7 9 9 78

Time Complexity:

Bubble sort:

```

for (i=0; i<n-1; i++) {
    for (j=i+1; j<n; j++) {
        if (arr[i] > arr[j]) {
            t = arr[i];
            arr[i] = arr[j];
            arr[j] = t;
        }
    }
}

```

Total T(n) = $(n-1) + (n-2) + \dots + 1$
 $= \frac{n(n-1)}{2} + n - 1$
 $= \frac{n^2 - n}{2} + n - 1$
 $= \frac{n^2 - n + 2n - 2}{2}$
 $= \frac{n^2 + n - 2}{2}$
 $= O(n^2)$

Selection sort:

```

for (i=0; i<n-1; i++) {
    min = i;
    for (j=i+1; j<n; j++) {
        if (arr[j] < arr[min]) {
            min = j;
        }
    }
    if (min != i) {
        t = arr[i];
        arr[i] = arr[min];
        arr[min] = t;
    }
}

```

Total T(n) = $(n-1) + (n-2) + \dots + 1$
 $= \frac{n(n-1)}{2} + n - 1$
 $= \frac{n^2 - n}{2} + n - 1$
 $= \frac{n^2 - n + 2n - 2}{2}$
 $= \frac{n^2 + n - 2}{2}$
 $= O(n^2)$

Insertion sort:

```

for (i=1; i<n; i++) {
    t = arr[i];
    j = i-1;
    while (j > 0 && arr[j] > t) {
        arr[j+1] = arr[j];
        j = j-1;
    }
    arr[j+1] = t;
}

```

Total T(n) = $(n-1) + (n-2) + \dots + 1$
 $= \frac{n(n-1)}{2} + n - 1$
 $= \frac{n^2 - n}{2} + n - 1$
 $= \frac{n^2 - n + 2n - 2}{2}$
 $= \frac{n^2 + n - 2}{2}$
 $= O(n^2)$

Merge sort:

```

MergeSort(arr, left, right) {
    if (left < right) {
        m = (left + right) / 2;
        MergeSort(arr, left, m);
        MergeSort(arr, m+1, right);
        Merge(arr, left, m, right);
    }
}

```

Total T(n) = $O(n) \times O(\log n)$
 $= O(n \log n)$

Quick sort:

```

QuickSort(arr, s, e) {
    if (s < e) {
        p = Partition(arr, s, e);
        QuickSort(arr, s, p-1);
        QuickSort(arr, p+1, e);
    }
}

```

Total T(n) = $O(n) \times O(\log n)$
 $= O(n \log n)$

$T(n) = T(n-1) + O(n)$
 $T(n-1) = T(n-2) + O(n-1)$
 $T(n) = T(n-2) + 2 + 3 + 4 + \dots + n$
 $T(n) = O(n^2)$

Conclusion: Hence, By Implementing and analysing of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort.

The time complexity plays an important role in the logical part of the program. And the Programs are successfully executed.



Marwadi
University
Marwadi Chandarana Group



FACULTY OF TECHNOLOGY

Department of AI, ML & DS

01AI0506 | DAA Lab Manual
